

```
#!/usr/bin/env python3
"""
DSR TOOLKIT - Practical Tools for Analyzing Double-Strand Resonance

This module provides ready-to-use functions for:
1. Loading and cleaning trajectory data
2. Fitting the DSR model
3. Visualizing results
4. Validating against theoretical predictions
5. Generating reports

Usage:
    from dsr_toolkit import DSRAnalyzer

    analyzer = DSRAnalyzer('trajectory_data.csv')
    results = analyzer.analyze()
    analyzer.plot()
    analyzer.report()
"""

import numpy as np
import pandas as pd
from scipy.optimize import minimize
import matplotlib.pyplot as plt
from dataclasses import dataclass
from typing import Tuple, Optional

@dataclass
class DSRParameters:
    """Container for DSR fit parameters"""
    k: float # Frequency
    A: float # Amplitude
    phi: float # Phase
    rms: float # RMS error
    k_deviation_pct: float
    A_deviation_pct: float

    @property
    def is_canonical(self) -> bool:
        """Check if parameters match canonical DSR"""
        return self.k_deviation_pct < 5 and self.A_deviation_pct < 10

    @property
```

```

def quality(self) -> str:
    """Return quality assessment"""
    if self.is_canonical:
        return "CANONICAL DSR"
    elif self.k_deviation_pct < 15 and self.A_deviation_pct < 25:
        return "MODERATE DSR"
    else:
        return "WEAK/ATYPICAL"

class DSRAnalyzer:
    """
    Complete toolkit for DSR analysis
    """

    # Theoretical values
    K_THEORY = 0.63
    A_THEORY = 0.35

    def __init__(self, data_path: Optional[str] = None, df: Optional[pd.DataFrame] = None):
        """
        Initialize analyzer with trajectory data

        Args:
            data_path: Path to CSV file with columns [traj_id, frame, x, y]
            df: Or provide DataFrame directly
        """
        if data_path:
            self.df = pd.read_csv(data_path)
        elif df is not None:
            self.df = df
        else:
            raise ValueError("Must provide either data_path or df")

        # Clean data
        self.df_clean = self.df.dropna(subset=['x', 'y'])
        self.x = self.df_clean['x'].values
        self.y = self.df_clean['y'].values

        # Results (populated after analysis)
        self.params: Optional[DSRParameters] = None

    def _loss_function(self, params: Tuple[float, float, float]) -> float:
        """
        DSR loss function:  $|x - 0.5| = |A * \cos(k*y + \phi)|$ 

        Returns mean squared error

```

```

        """
        k, A, phi = params
        x_centered = self.x - 0.5
        model = np.abs(A * np.cos(k * self.y + phi))
        return np.mean((np.abs(x_centered) - model)**2)

def fit_dsr(self,
            k_init: float = 0.63,
            A_init: float = 0.35,
            phi_init: float = 0.0) -> DSRParameters:
    """
    Fit DSR model to data

    Args:
        k_init: Initial guess for frequency
        A_init: Initial guess for amplitude
        phi_init: Initial guess for phase

    Returns:
        DSRParameters object with fitted values
    """
    x0 = [k_init, A_init, phi_init]
    bounds = ((0.1, 1.5), (0.05, 0.5), (-np.pi, np.pi))

    result = minimize(
        self._loss_function,
        x0,
        bounds=bounds,
        method='L-BFGS-B'
    )

    k_opt, A_opt, phi_opt = result.x
    rms = np.sqrt(result.fun)

    k_dev = abs(k_opt - self.K_THEORY) / self.K_THEORY * 100
    A_dev = abs(A_opt - self.A_THEORY) / self.A_THEORY * 100

    self.params = DSRParameters(
        k=k_opt,
        A=A_opt,
        phi=phi_opt,
        rms=rms,
        k_deviation_pct=k_dev,
        A_deviation_pct=A_dev
    )

    return self.params

```

```

def generate_strands(self,
                    y_range: Optional[Tuple[float, float]] = None,
                    n_points: int = 1000,
                    use_fit: bool = True) -> Tuple[np.ndarray, np.ndarray, np
"""
Generate theoretical or fitted strand coordinates

Args:
    y_range: (y_min, y_max) or None for auto
    n_points: Number of points to generate
    use_fit: If True, use fitted params; else use theory

Returns:
    (y, strand_0, strand_1) arrays
"""
if y_range is None:
    y_range = (0, self.y.max())

y = np.linspace(y_range[0], y_range[1], n_points)

if use_fit and self.params:
    k, A, phi = self.params.k, self.params.A, self.params.phi
else:
    k, A, phi = self.K_THEORY, self.A_THEORY, 0.0

strand_0 = 0.5 + A * np.cos(k * y + phi)
strand_1 = 0.5 - A * np.cos(k * y + phi)

return y, strand_0, strand_1

def calculate_residuals(self) -> np.ndarray:
"""
Calculate distance of each point to nearest strand

Returns:
    Array of residual distances
"""
if self.params is None:
    raise ValueError("Must call fit_dsr() first")

k, A, phi = self.params.k, self.params.A, self.params.phi

residuals = []
for x, y in zip(self.x, self.y):
    strand_0 = 0.5 + A * np.cos(k * y + phi)
    strand_1 = 0.5 - A * np.cos(k * y + phi)

```

```

        dist_0 = abs(x - strand_0)
        dist_1 = abs(x - strand_1)
        residuals.append(min(dist_0, dist_1))

    return np.array(residuals)

def plot(self, figsize: Tuple[int, int] = (16, 10), save_path: Optional[str]
        """
        Create comprehensive visualization of DSR analysis

        Args:
            figsize: Figure size (width, height)
            save_path: Optional path to save figure
        """
        if self.params is None:
            raise ValueError("Must call fit_dsr() first")

        fig, axes = plt.subplots(2, 2, figsize=figsize)

        # Generate strands
        y_theory, s0_theory, s1_theory = self.generate_strands(use_fit=False)
        y_fit, s0_fit, s1_fit = self.generate_strands(use_fit=True)

        # Plot 1: Data vs Theory
        ax = axes[0, 0]
        ax.scatter(self.x, self.y, alpha=0.5, s=20, c='cyan',
                   edgecolors='navy', linewidths=0.5, label='Data')
        ax.plot(s0_theory, y_theory, 'r-', lw=2.5, alpha=0.8, label='Theory S0')
        ax.plot(s1_theory, y_theory, 'b-', lw=2.5, alpha=0.8, label='Theory S1')
        ax.axvline(0.5, color='orange', ls='--', lw=2, alpha=0.7, label='Critical')
        ax.set_xlabel('Re(z)', fontweight='bold')
        ax.set_ylabel('Im(z)', fontweight='bold')
        ax.set_title(f'Theoretical DSR\n(k={self.K_THEORY}, A={self.A_THEORY})',
                     fontweight='bold')
        ax.legend()
        ax.grid(alpha=0.3)
        ax.set_xlim(-0.05, 1.05)

        # Plot 2: Data vs Fit
        ax = axes[0, 1]
        ax.scatter(self.x, self.y, alpha=0.5, s=20, c='cyan',
                   edgecolors='navy', linewidths=0.5, label='Data')
        ax.plot(s0_fit, y_fit, 'r-', lw=2.5, alpha=0.8, label='Fit S0')
        ax.plot(s1_fit, y_fit, 'b-', lw=2.5, alpha=0.8, label='Fit S1')
        ax.axvline(0.5, color='orange', ls='--', lw=2, alpha=0.7, label='Critical')
        ax.set_xlabel('Re(z)', fontweight='bold')

```

```

ax.set_ylabel('Im(z)', fontweight='bold')
ax.set_title(f'Fitted DSR\n(k={self.params.k:.4f}, A={self.params.A:.4f})
              fontweight='bold')
ax.legend()
ax.grid(alpha=0.3)
ax.set_xlim(-0.05, 1.05)

# Plot 3: Residuals
ax = axes[1, 0]
residuals = self.calculate_residuals()
ax.scatter(self.y, residuals, alpha=0.6, s=15, c='purple',
           edgecolors='black', linewidths=0.3)
ax.axhline(self.params.rms, color='red', ls='--', lw=2,
           label=f'RMS={self.params.rms:.4f}')
ax.set_xlabel('Im(z)', fontweight='bold')
ax.set_ylabel('Distance to Nearest Strand', fontweight='bold')
ax.set_title('Residual Analysis', fontweight='bold')
ax.legend()
ax.grid(alpha=0.3)

# Plot 4: Distribution
ax = axes[1, 1]
ax.hist(self.x, bins=40, alpha=0.7, color='cyan', edgecolor='black', lw=1)
ax.axvline(0.5, color='orange', ls='--', lw=2.5, label='x=0.5')
ax.axvline(0.5 - self.params.A, color='blue', ls=':', lw=2, label='Left m
ax.axvline(0.5 + self.params.A, color='red', ls=':', lw=2, label='Right m
ax.set_xlabel('Re(z)', fontweight='bold')
ax.set_ylabel('Frequency', fontweight='bold')
ax.set_title('Distribution of Points', fontweight='bold')
ax.legend()
ax.grid(alpha=0.3, axis='y')

plt.tight_layout()

if save_path:
    plt.savefig(save_path, dpi=150, bbox_inches='tight', facecolor='white

return fig

def report(self) -> str:
    """
    Generate text report of analysis

    Returns:
        Formatted report string
    """
    if self.params is None:

```

```

        raise ValueError("Must call fit_dsr() first")

    left = (self.x < 0.5).sum()
    right = (self.x >= 0.5).sum()
    total = len(self.x)

    report = f"""
{'='*70}

                        DSR ANALYSIS REPORT
{'='*70}

📊 DATA SUMMARY:
    Total points:           {total}
    Unique trajectories: {self.df_clean['traj_id'].nunique()}
    Y-range:                [{self.y.min():.4f}, {self.y.max():.4f}]
    X-range:                [{self.x.min():.4f}, {self.x.max():.4f}]

✨ FIT RESULTS:
    k (frequency):  {self.params.k:.6f}   [Theory: {self.K_THEORY:.6f}]
    A (amplitude):  {self.params.A:.6f}   [Theory: {self.A_THEORY:.6f}]
    φ (phase):      {self.params.phi:.6f}
    RMS error:      {self.params.rms:.6f}

🔧 DEVIATIONS:
    k deviation: {self.params.k_deviation_pct:.2f}%
    A deviation: {self.params.A_deviation_pct:.2f}%

🎯 VALIDATION: {self.params.quality}

📊 STRAND DISTRIBUTION:
    Left strand (x < 0.5):  {left:4d} points ({left/total*100:5.1f}%)
    Right strand (x ≥ 0.5): {right:4d} points ({right/total*100:5.1f}%)

{'='*70}

    """

    return report

def analyze(self) -> DSRParameters:
    """
    Run complete analysis pipeline

    Returns:
        DSRParameters with results
    """
    print("Running DSR analysis...")
    params = self.fit_dsr()

```

```

        print(self.report())
        return params

# Convenience function for quick analysis
def quick_analyze(data_path: str, plot: bool = True, save: bool = False):
    """
    Quick DSR analysis from CSV file

    Args:
        data_path: Path to trajectory CSV
        plot: Whether to generate plots
        save: Whether to save plot to file

    Returns:
        DSRAnalyzer instance with results
    """
    analyzer = DSRAnalyzer(data_path)
    analyzer.analyze()

    if plot:
        save_path = data_path.replace('.csv', '_dsr_analysis.png') if save else None
        analyzer.plot(save_path=save_path)
        if not save:
            plt.show()

    return analyzer

if __name__ == "__main__":
    print(__doc__)
    print("\nExample usage:")
    print("  analyzer = DSRAnalyzer('my_data.csv')")
    print("  results = analyzer.analyze()")
    print("  analyzer.plot(save_path='dsr_results.png')")

```